



UNIVERSITÀ DEGLI STUDI DI SALERNO
DIPARTIMENTO DI INFORMATICA



CORSO DI GRAFICA E INTERATTIVITA'

2024/2025

Game Engine

Lezione 1: Introduzione a Unity3D

Docenti:

Prof. Andrea F. Abate

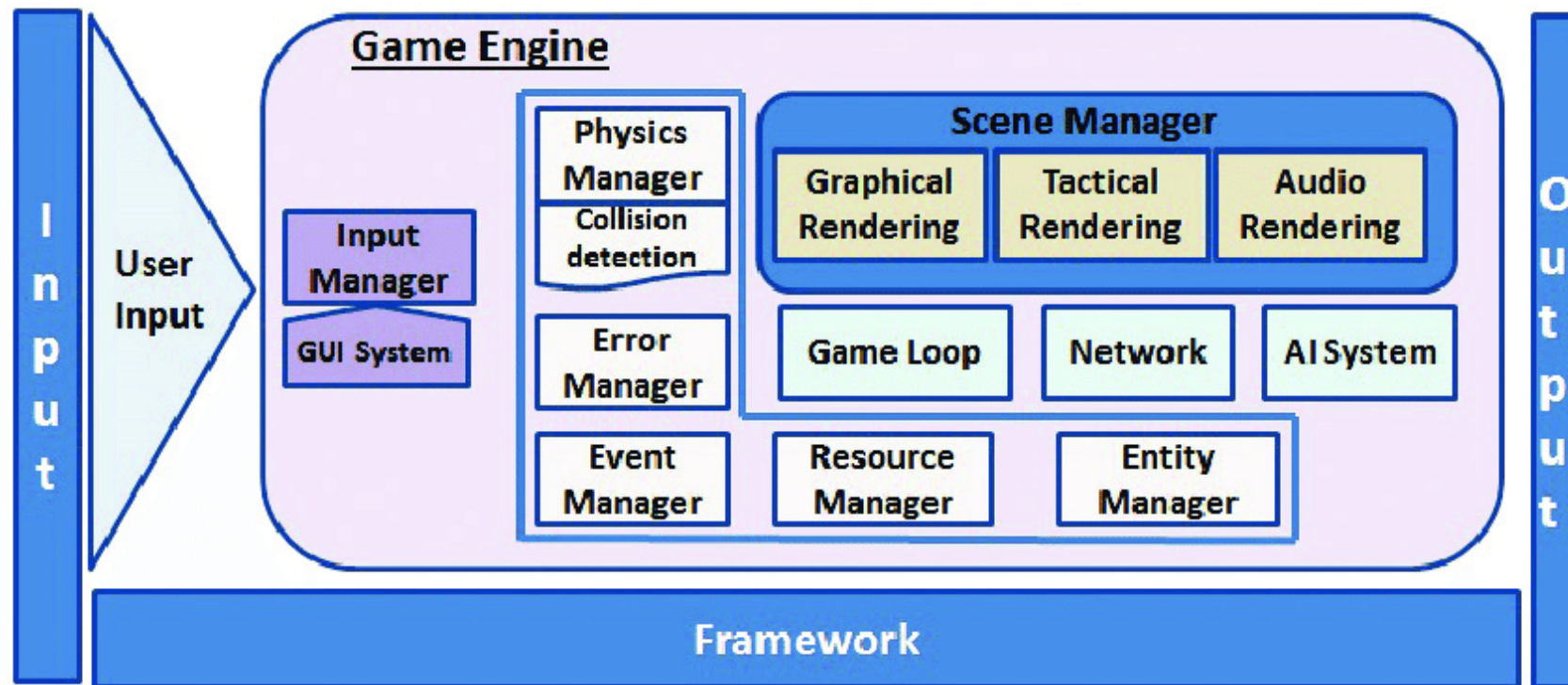
Dott.ssa Lucia Cimmino



What is a 3D game engine?

- A **Game Engine** is the core software of a video game or any real-time graphics application. It provides the basic technologies needed, simplifies development, and allows the game to run on different platforms.
- Basic functionality typically provided by a Game Engine:
 - A **rendering engine** (*Renderer or Graphic Engine*) for 2D and 3D graphics
 - A **physics engine** or collision detection system
 - Audio engine
 - **Scripting system**
 - **Asset Management**
 - **Animations**
 - Networking
 - Artificial intelligence
 - A scene graph

What is a 3D game engine?



Why using game engine?

- Simplification of Development:**

The integration of standard components allows developers to focus on the game's unique aspects, saving time and resources.

- Modularity and Reusability:**

A modular structure enables the reuse of components and scripts across different projects, enhancing efficiency and productivity.

- Rapid Prototyping:**

It facilitates the quick creation of prototypes and testing of new ideas, which is fundamental in academic and experimental contexts.

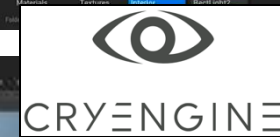
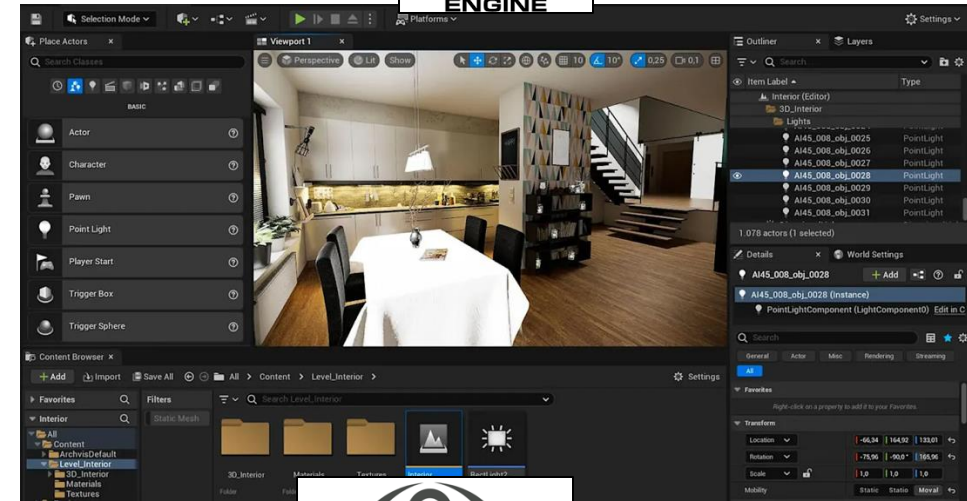
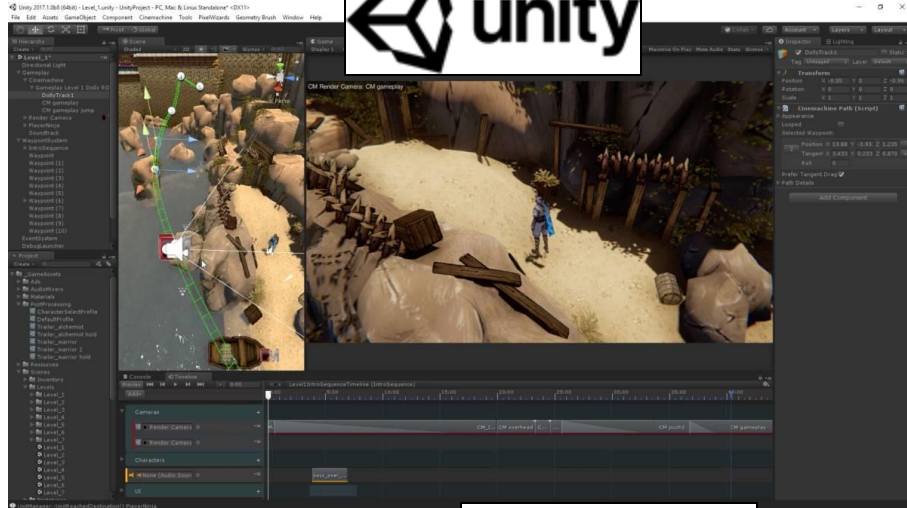
- Optimization and Performance:**

Modern engines include profiling and optimization tools, ensuring high performance even in complex projects.

- Multiplatform Support:**

They simplify deployment across various platforms (PC, consoles, mobile, VR/AR), broadening the potential audience of the final product.

Different game engine



Different game engine

Engine	Scripting Languages	Licence	Supported Platforms	Key features
Unity	C#	Proprietary, freemium	PC, console, mobile, web, VR/AR	Large community, Asset Store, intuitive interface, scalability
Unreal Engine	C++ and Blueprint (visual scripting)	Proprietary, royalty-based	PC, console, mobile, VR/AR	High fidelity graphics, advanced physics engine, ideal for AAA
Godot	GDScript (similar to Python) and C#	Open Source (MIT License)	PC, console, mobile, web	Lightweight, flexible, native 2D/3D support, growing community
CryEngine	C++ and visual scripting	Proprietary, free with royalties	PC, console	Realistic graphics, advanced physics, suitable for AAA games

Main Terms and Concepts

Before any 3D Engine...

DCC

- **Digital Content Creation applications**
- These programs include 3ds Max, Maya Cinema4D, Blender, and many others.
- They DO NOT necessarily feature a 3DEngine!

Frame rate Per Second (FPS)

- The frame rate, measured in frames per second (FPS), is the frequency at which images (frames) are displayed in a video, animation, game etc.
- Higher frame rates (e.g., 60 fps or more) result in smoother, more responsive gameplay, enhances immersion and reduces input lag:
 - **24 fps:** Standard for cinematic content.
 - **30 fps:** Common in video and some casual games.
 - **60 fps:** Preferred in fast-paced or competitive gaming for better fluidity and responsiveness.



FPS Vs FPS



Frame rate Per Second

VS

First Person Shooter



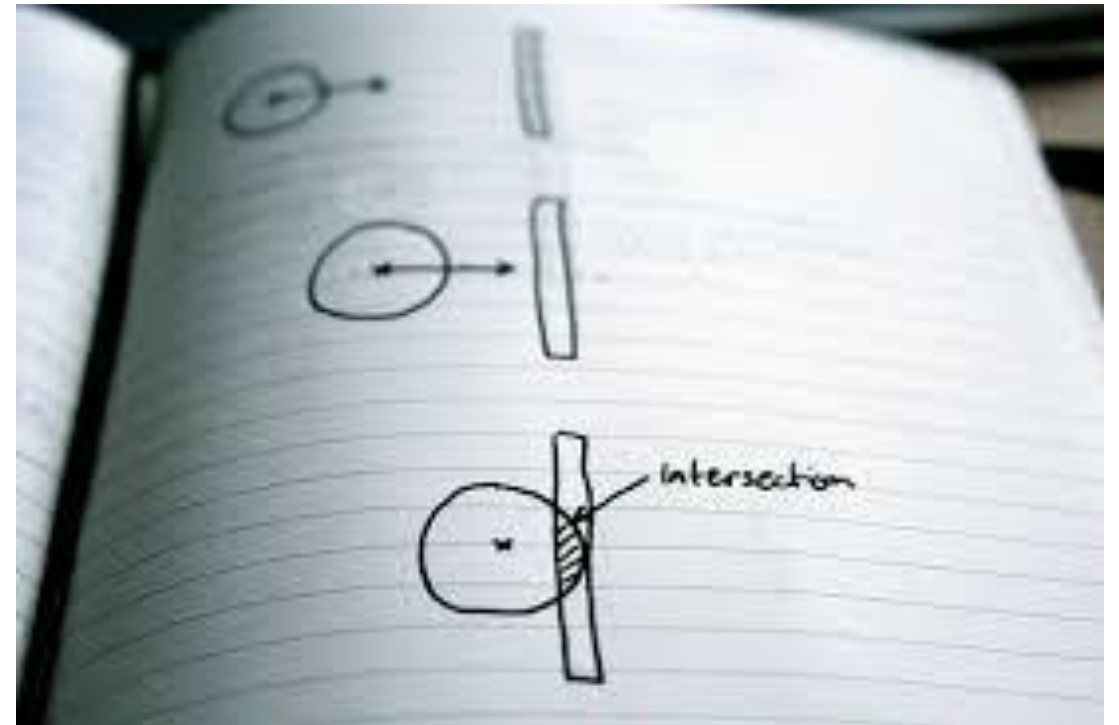
Navigation

- One of the big differences between DCC and 3D Engine is scene navigation.
- A big part of games is the method of navigating through the 3D environment.
- The type of game plays a big part in the choice, but, additionally, the skill level of the player also dictates the choices.
- The first big question always is : First Person or Third Person View?



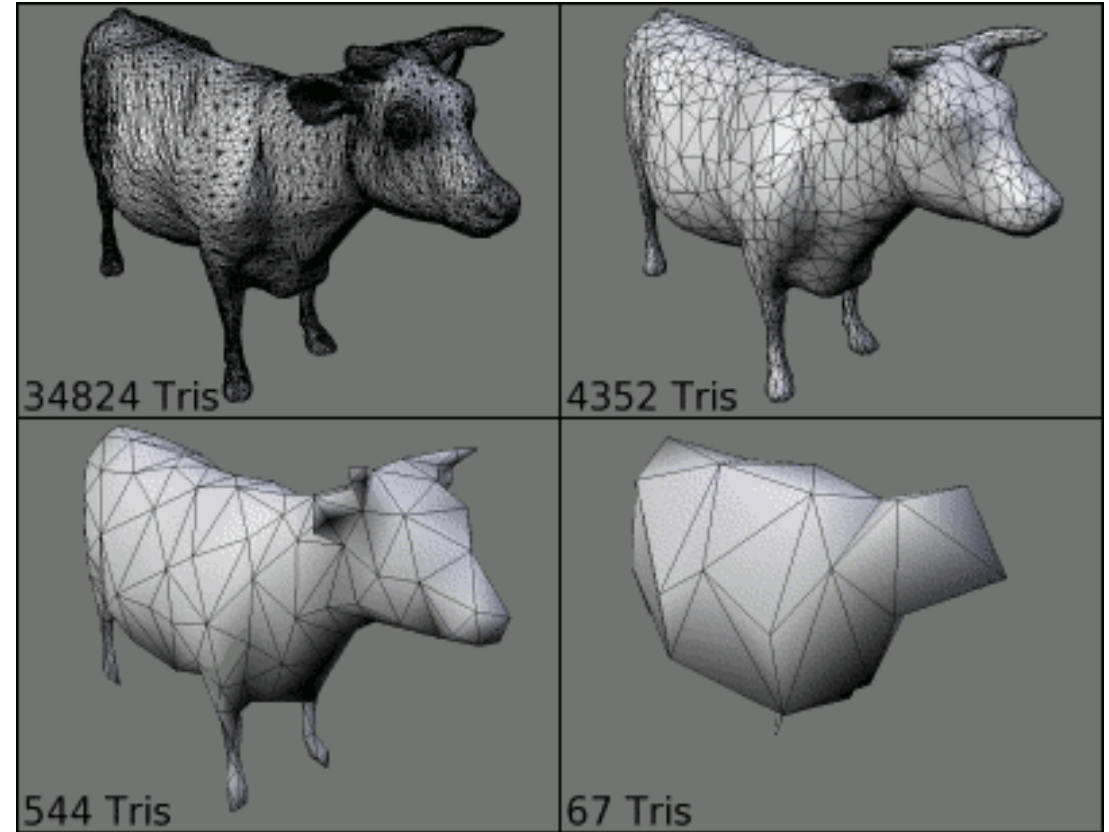
Collisions

- Once players are allowed to travel wherever they wish in a 3D world, physical constraints must be set to prevent them from traveling through walls.
- Yet, allowing them to move over uneven terrain, stairs, and other obstacles.



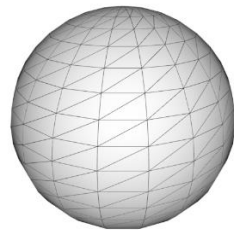
Polygons

- Polygons are simple shapes used to build 3D objects. In game engines, these are usually triangles.
- Objects are created by connecting a certain number of triangles (forming a **mesh**).
- More polygons are used more detailed will be the 3D object
- Using too many triangles can slow down a game, so it's important to find a balance between visual quality and smooth gameplay.

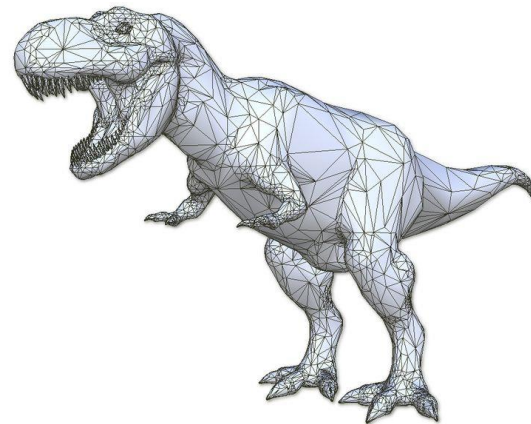


Primitive and Mesh Object

- A primitive is a basic object that is parametrically generated by a set of parameters:
 - A sphere, for example, can be generated by its radius and height and width segments .
- A mesh object is defined by giving the exact location, or offset, of its vertices in relation to the object's transform axis.



VS



Balancing details and performance: Level Of Detail (LOD)

- Using too many polygons can lead to slowdowns, especially in real-time applications such as video games.
- It is important to optimize the number of polygons.
- **LOD** methods dynamically adjust the number of polygons based on the object's distance from the camera, ensuring detailed visuals up close while simplifying distant objects to maintain smooth performance.



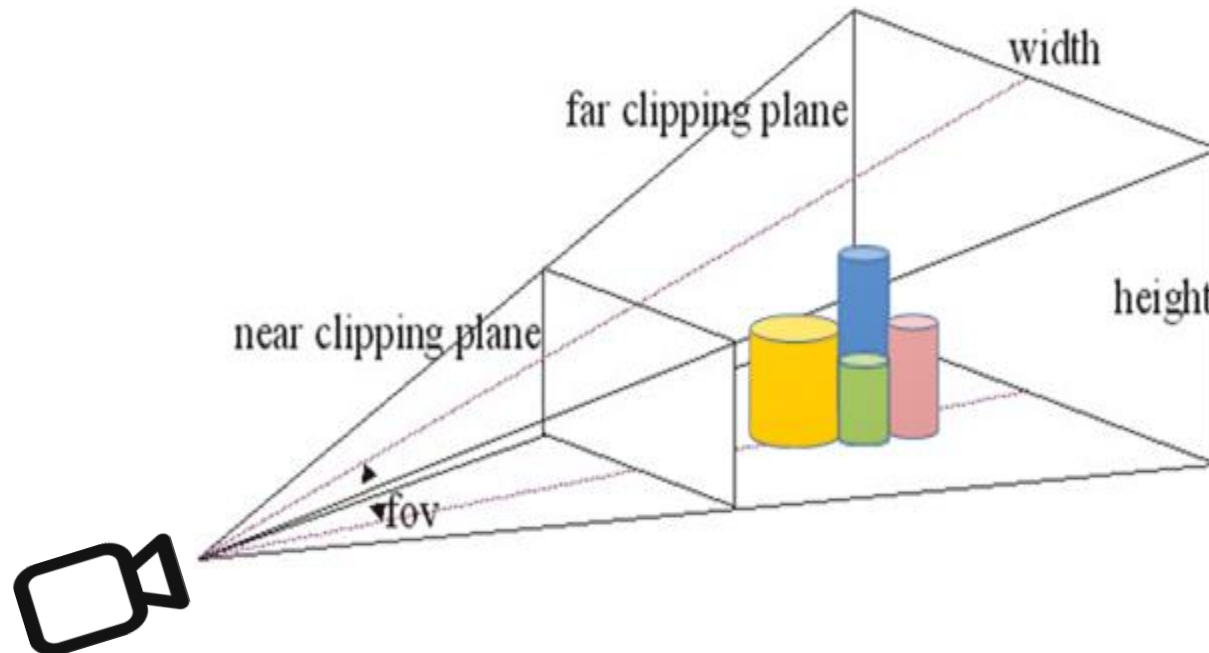
Distance Culling and Fog

- Related to LOD, **distance culling** means that at a particular distance, the object will no longer be drawn in the scene.
- When a high-poly-count object takes only a few pixels onscreen, it is wasting resources.
- Typically, one might use a low-poly stand-in at a particular distance, then use distance cull to stop drawing the low-poly stand-in at some point.
- Sometimes the **Fog** helps to create a good effect



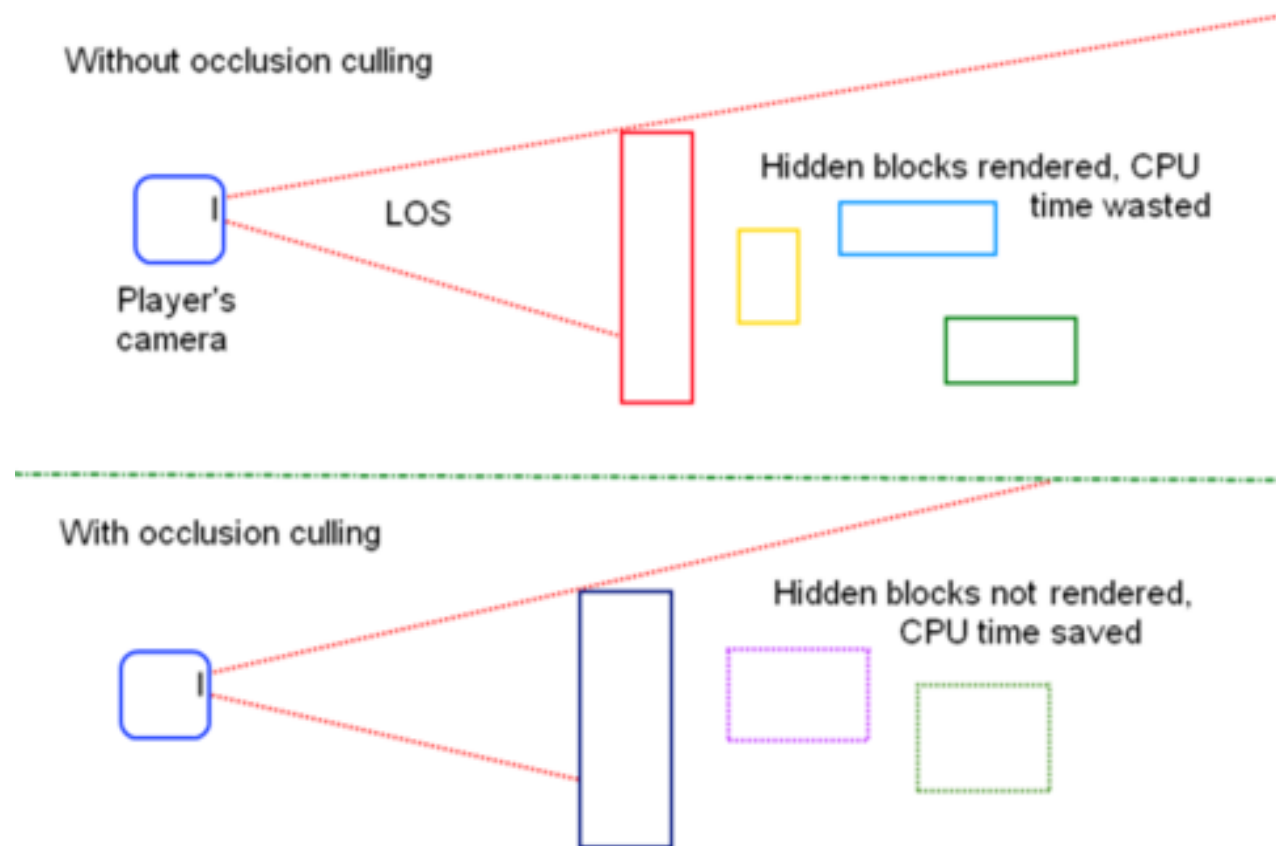
Plane Clipping

- Cameras have near and far clipping planes
- Objects are not rendered if they are closer than the near clipping plane or beyond the far clipping plane.



Occlusion Culling

Another rendering optimization technique that prevents the engine from drawing objects that are hidden behind others



Getting Started with





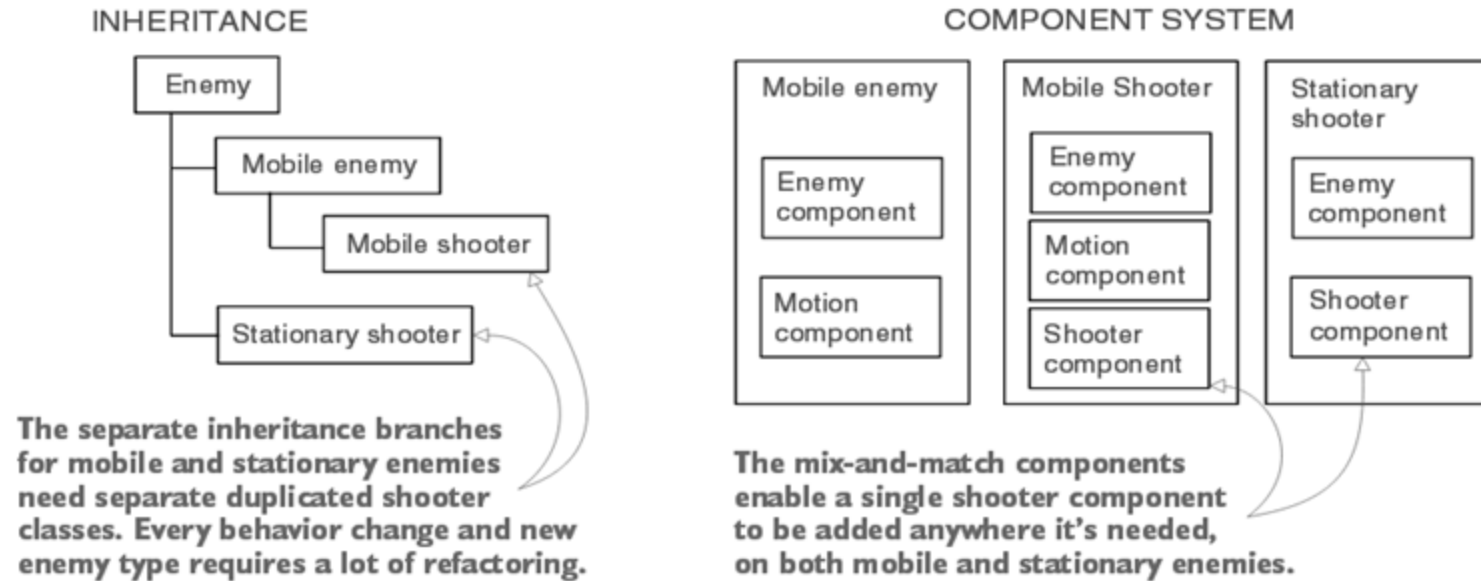
Why Unity?

Engine	Scripting Languages	Licence	Supported Platforms	Key features
Unity	C#	Proprietary, freemium	PC, console, mobile, web, VR/AR	Large community, Asset Store, intuitive interface, scalability

- Unity is a professional-quality game engine used to create video games targeting a variety of platforms.
- Unity has physics simulation, normal maps, screen space ambient occlusion (SSAO), dynamic shadows...and the list goes on.
- Unity's toolset is a high degree of cross-platform support.
- A third and subtler benefit comes from the modular component system used to construct game objects.
- Beginner-friendly with an intuitive interface for the development workflow.
- Large community and extensive documentation

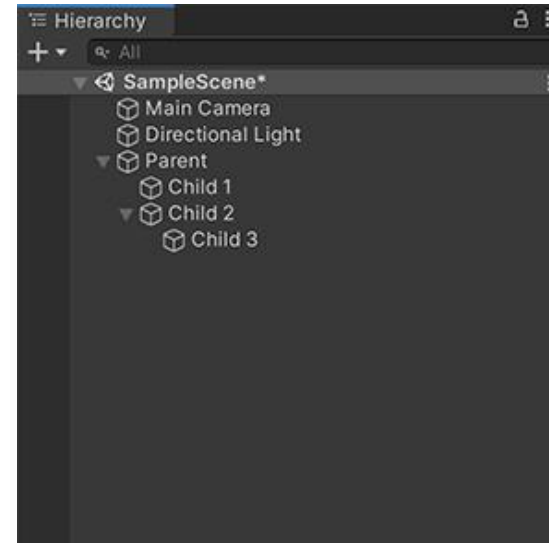
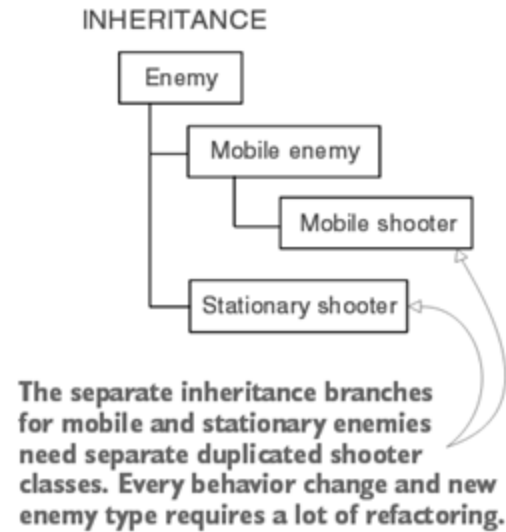


Components building



- Unity use a **component-based system** for the game logic.
- This arrangement allows for fast and flexible prototyping. Instead of modifying complex class hierarchies, developers can quickly add, remove, or adjust components to change an object's behaviour.
- This eliminates the need for refactoring inheritance chains, making game development more efficient and modular.

Inheritance in Unity



📌 But does inheritance exist in Unity?

- Unity allows **parent-child relationships** between GameObjects in the **Hierarchy View**, which helps with scene organization.
- Components themselves are C# classes that can inherit from MonoBehaviour, so inheritance exists, but it is only used within individual components and not as a basis for structuring game objects.

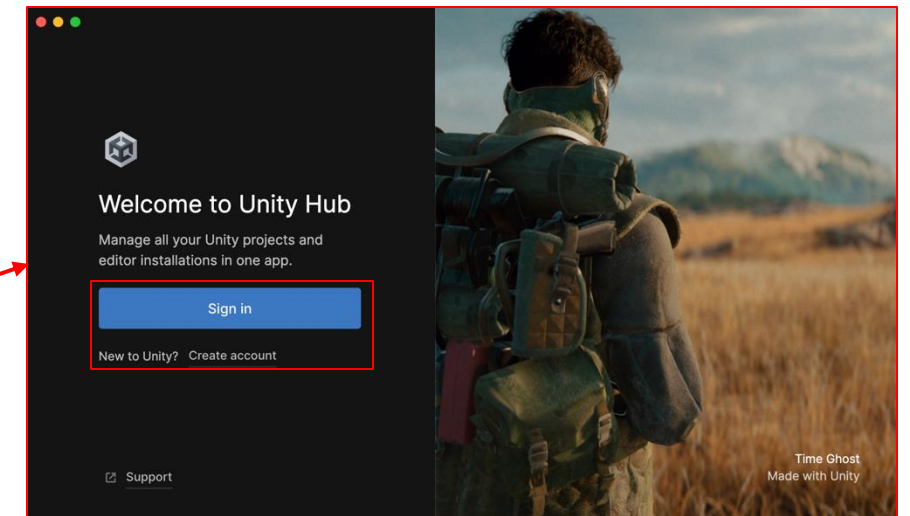
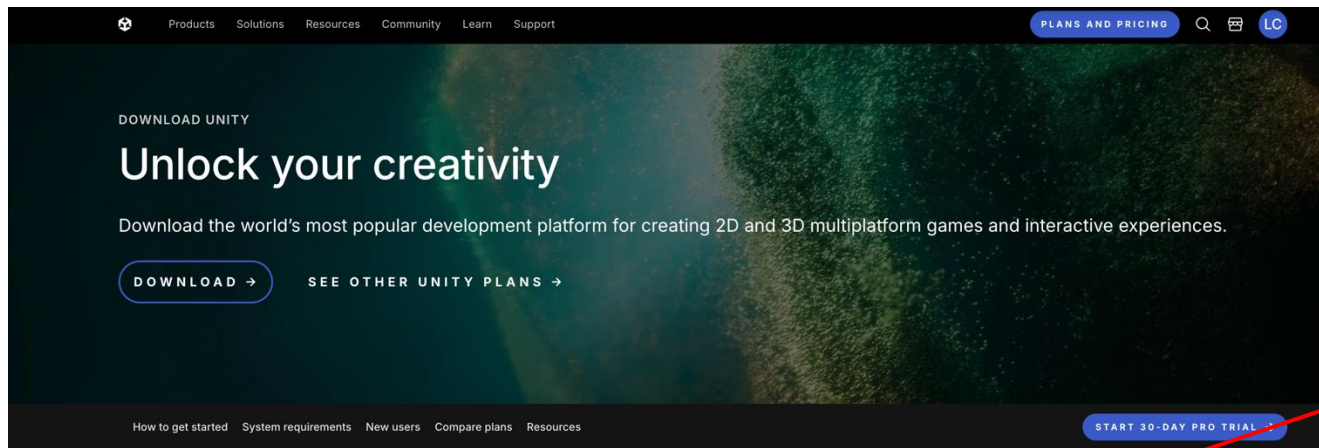
Unity installation





Setting up Unity

- Visit <https://unity.com/download> and download **Unity Hub**.
- Sign in or create an account



Create with Unity in three steps

1. Download the Unity Hub

Follow the instructions onscreen for guidance through the installation process and setup.

[Download for Windows](#)

[Download for Mac](#)

[Instructions for Linux](#)

2. Choose your Unity version

Install the latest version of Unity, an older release, or a beta featuring the latest in-development features.

[Visit the download archive](#)

3. Start your project

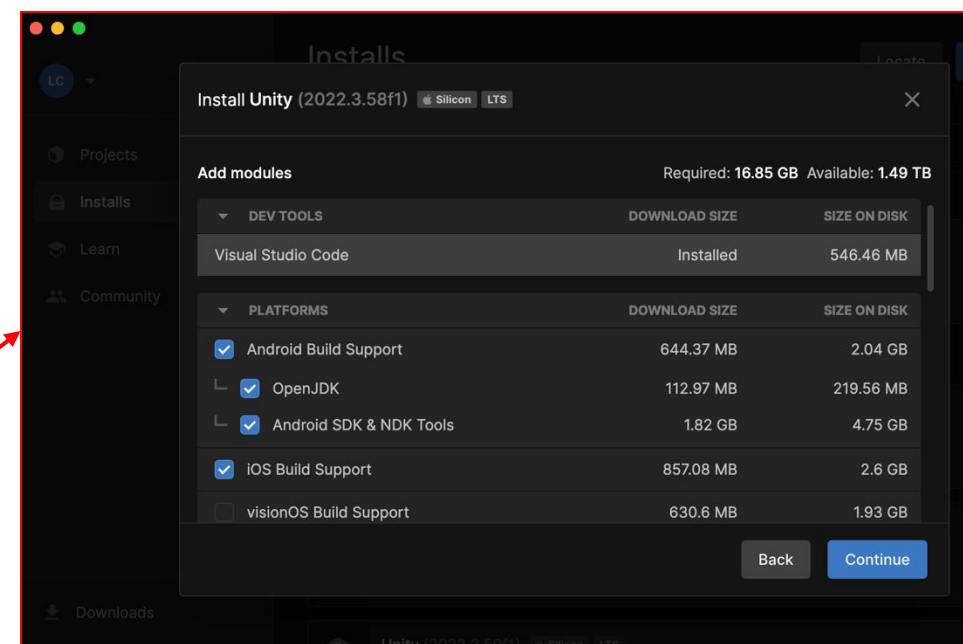
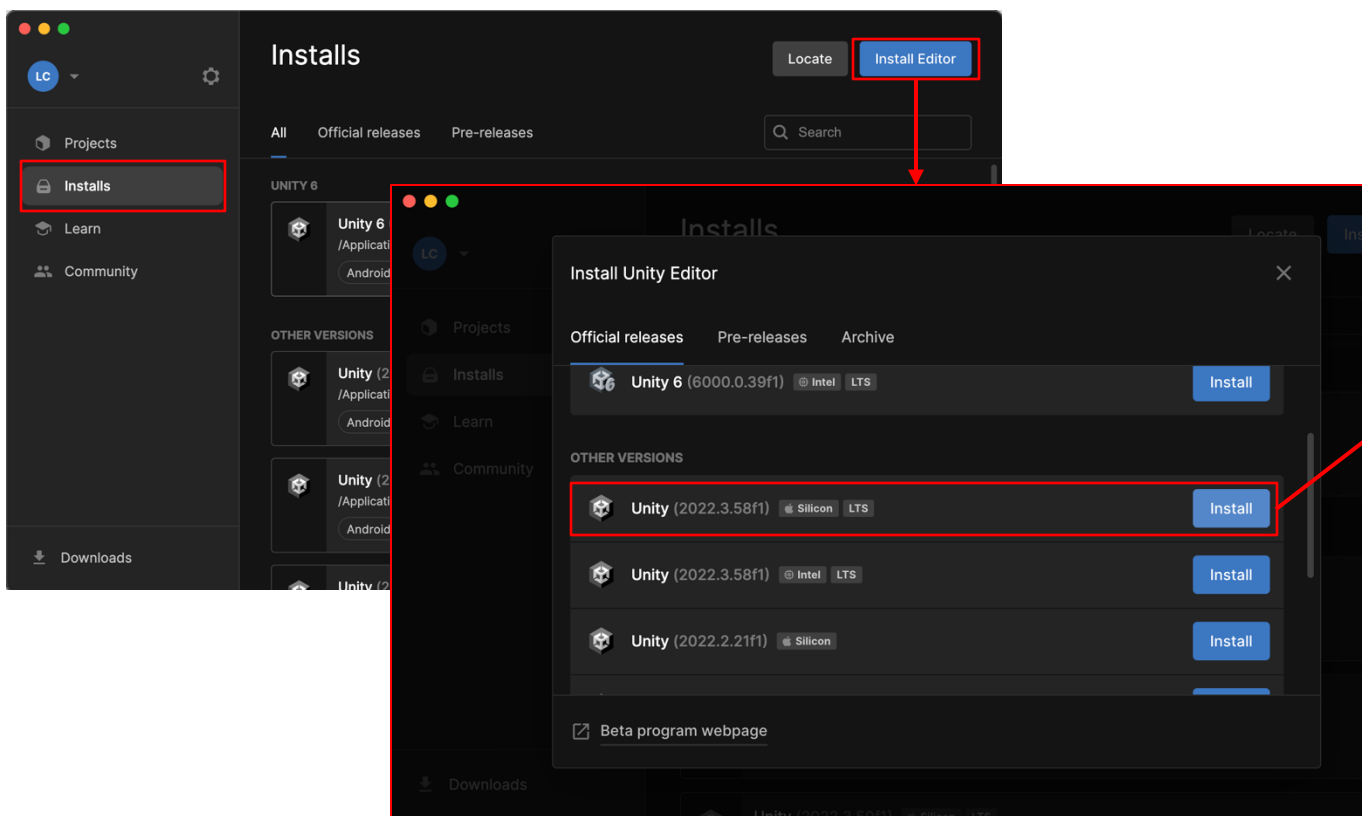
Begin creating from scratch, or pick a template to get your first project up and running quickly. Access tutorial videos designed to support creators, from beginners to experts.

[Access our Pro Onboarding Guide](#)



Unity LTS

- From the Unity hub install a Unity version LTS (Long Term Support) recommended for stability.
- **Recommended version** for the course activities and exam: **2022.3.58f1**



Unity Versioning System: Unity 6

- Unity has transitioned from the **year-based versioning system** (e.g., Unity 2022 LTS) to a **sequential numbering system** (Unity 6).
- **Unity 6** is the latest version of Unity and includes all the features from Unity 6 Preview, 2023.2, and 2023.1.
- New Features and Improvements:
 - Rendering and Performance.
 - Multiplayer Tools.
 - Artificial Intelligence Support.

Unity 2022 (LST) VS Unity 6

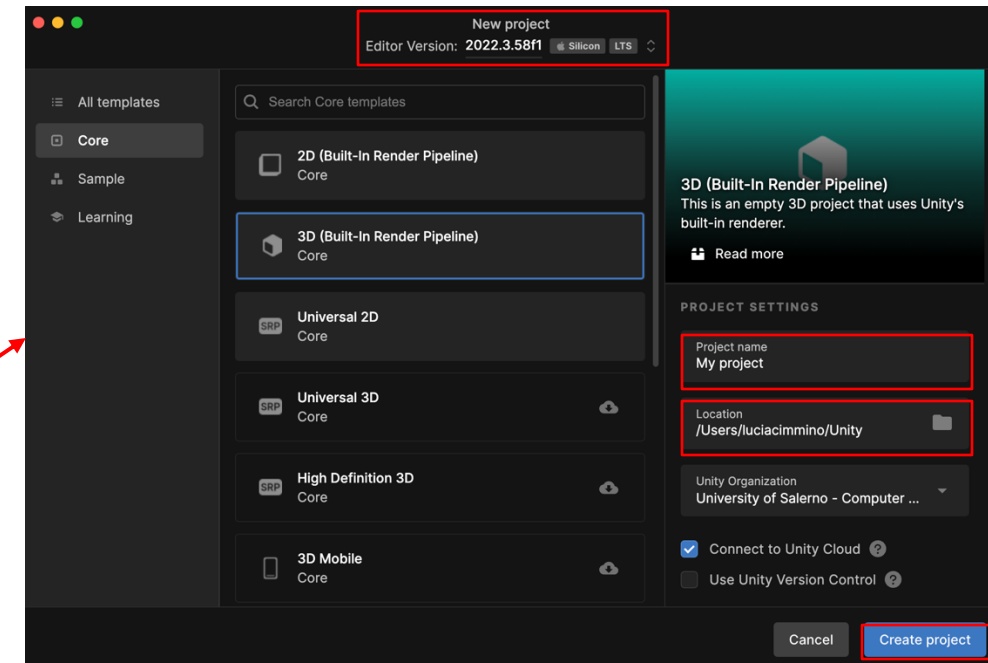
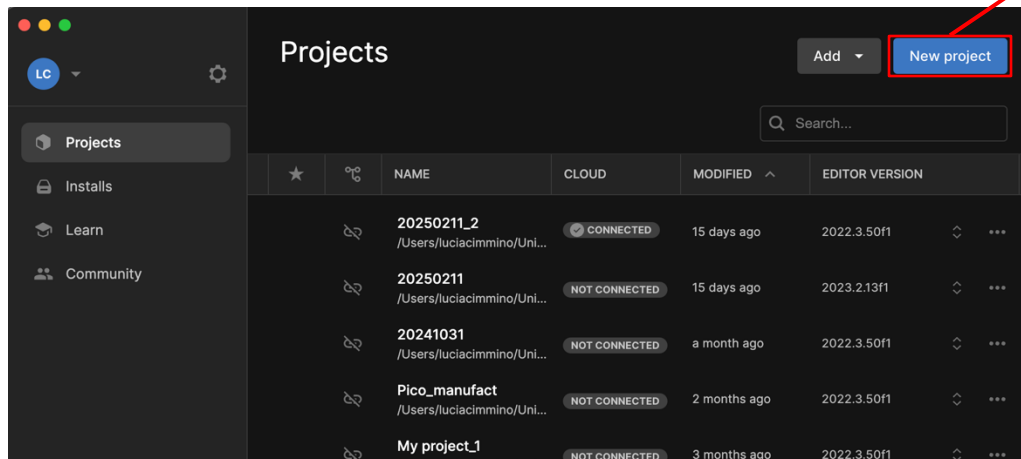
- Stability vs. New Features:
 - Unity 2022 LTS is optimized for reliability and long-term support.
 - Unity 6 introduces new rendering, AI, and multiplayer features.
- Performance:
 - Unity 2022 LTS may offer better physics performance in some cases.
 - Unity 6 improves rendering pipelines but may require adjustments when importing older assets.
- Choosing the Right Version:
 - Use Unity 2022 LTS for projects that need a stable and well-tested environment.
 - Use Unity 6 if you want access to the latest features and are open to potential adjustments in workflow.

Unity Interface

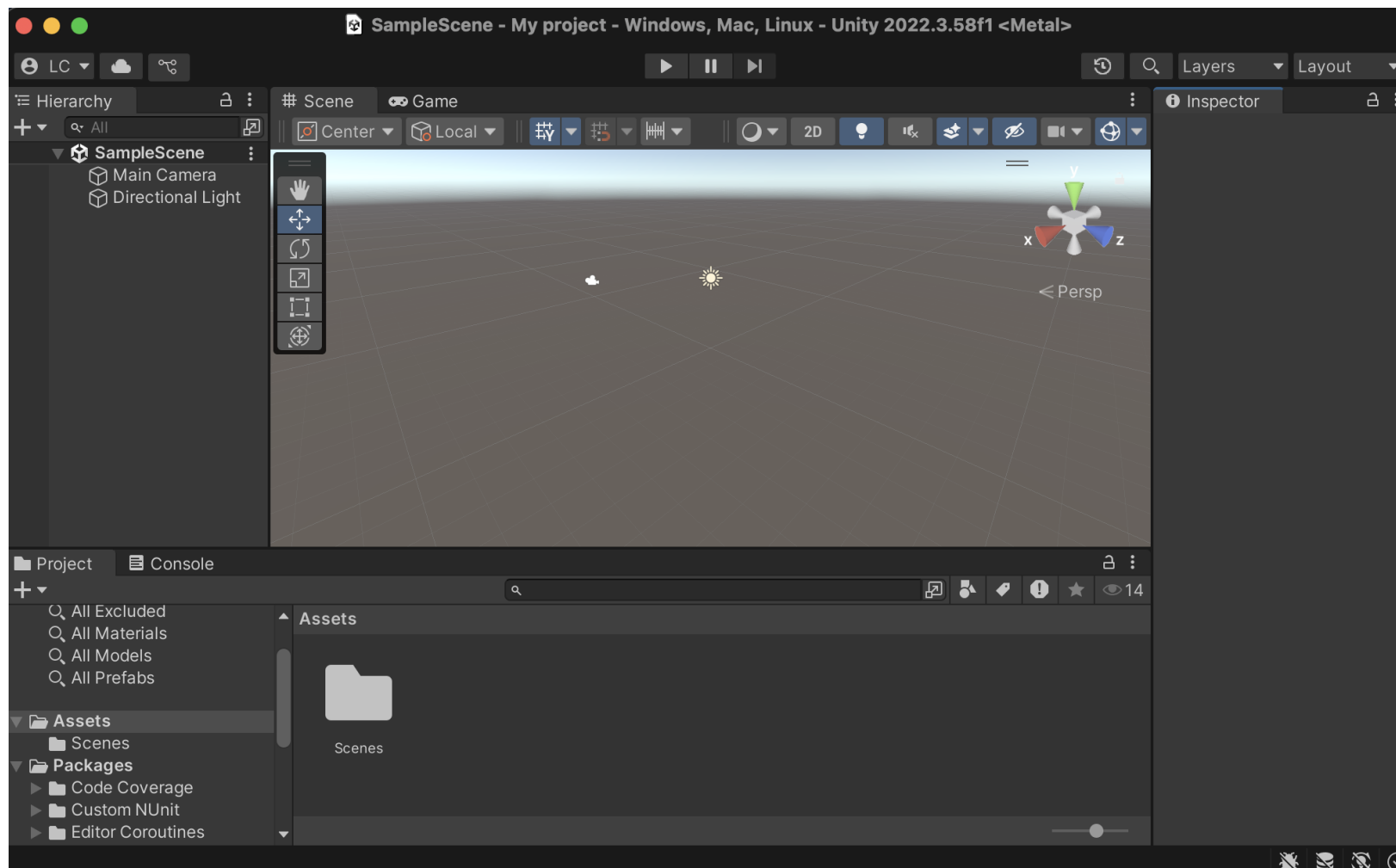


Create a project

- Open Unity Hub and go to the Projects tab.
- Click "New Project".
- Select the 3D (core) Template.
- Check the Unity version at the top of the interface.
- Name your project.
- Choose the save location for your project.
- Click "Create Project" to start working in Unity.



Create a project





(A) The Toolbar provides access to your Unity Account and Unity Cloud Services. It also contains controls for Play mode; Undo history; Unity Search; a layer visibility menu; and the Editor layout menu.

(B) The Hierarchy window is a hierarchical text representation of every GameObject in the Scene. Each item in the Scene has an entry in the hierarchy, so the two windows are inherently linked. The hierarchy reveals the structure of how GameObjects attach to each other.

(C) The Game view simulates what your final rendered game will look like through your Scene Cameras. When you click the Play button, the simulation begins.

(D) The Scene view allows you to visually navigate and edit your Scene. The Scene view can display a 3D or 2D perspective, depending on the type of Project you are working on.



(E) Overlays contain the basic tools for manipulating the Scene view and the GameObjects within it. You can also add custom Overlays to improve your workflow.

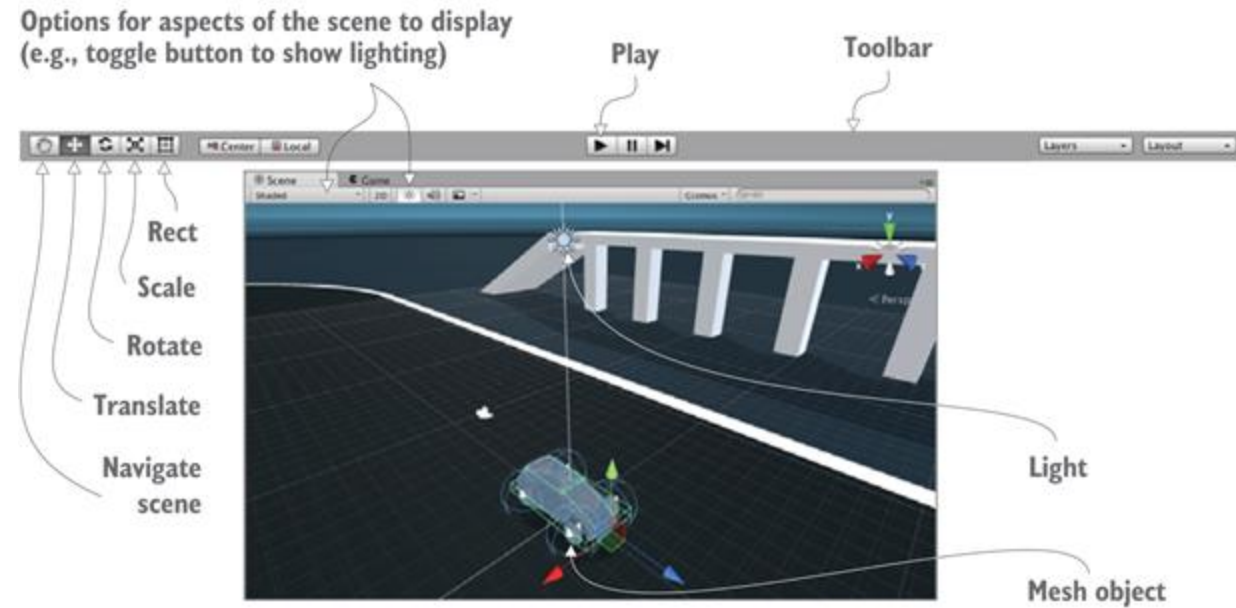
(F) The Inspector window allows you to view and edit all the properties of the currently selected GameObject. Because different types of GameObjects have different sets of properties, the layout and contents of the Inspector window change each time you select a different GameObject.

(G) The Project window displays your library of Assets that are available to use in your Project. When you import Assets into your Project, they appear here.

(H) The status bar provides notifications about various Unity processes, and quick access to related tools and settings.

Scene View

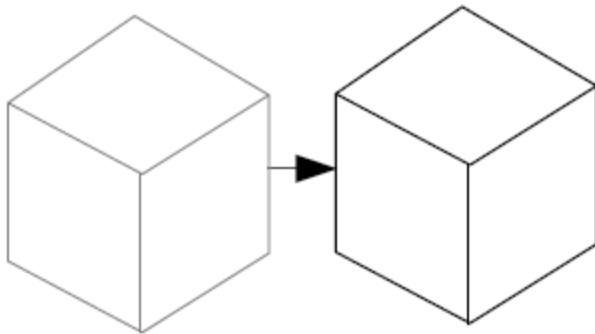
- This is where you can see what the game world looks like and move objects around.
- Mesh objects in the scene appear as, well, the mesh object (defined in a moment).
- You can also see a number of other objects in the scene, represented by various icons and coloured lines: cameras, lights, audio sources, collision regions, and so forth.
- Note that the Scene view isn't the same as the view in the running game. Here you're able to look around the scene at will without being constrained to the game's view.



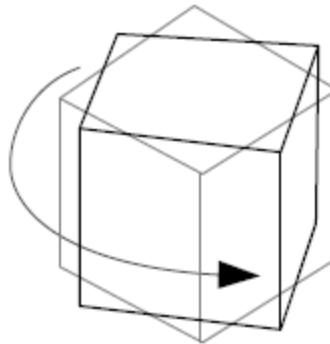
Navigate in the Scene

- **Translate:** move object along axis
- **Rotate:** rotate object along axis
- **Scale:** change size of the object along axis

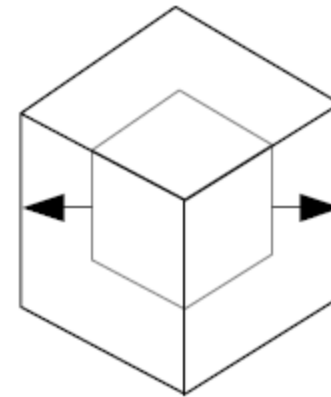
Translate



Rotate

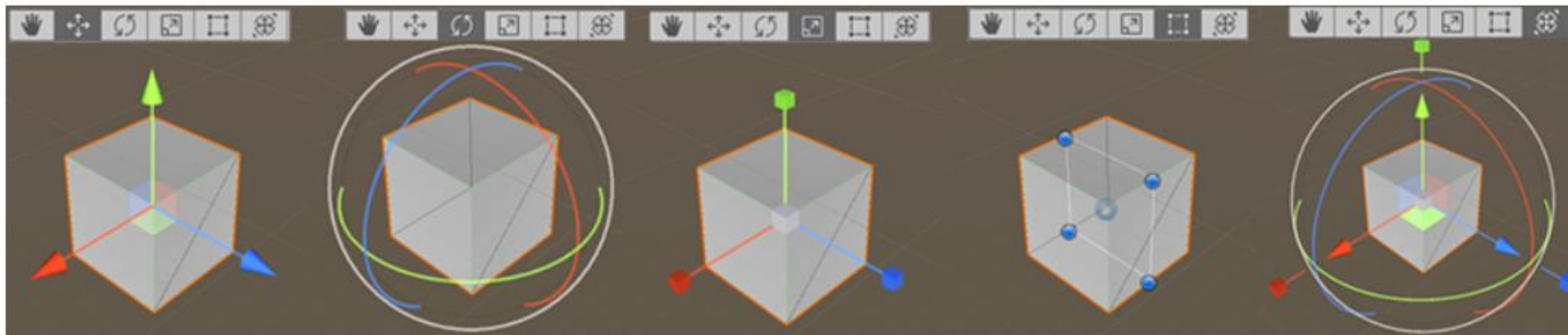


Scale



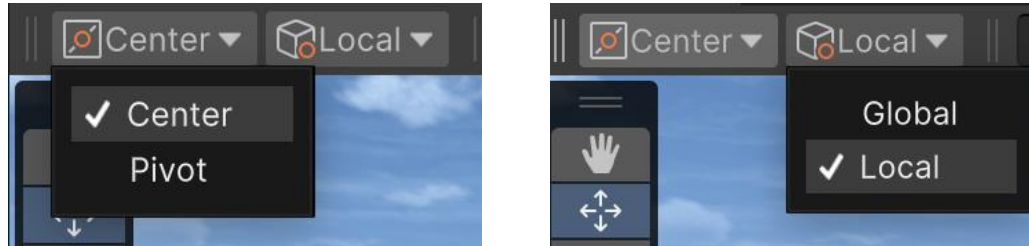
Transform Gizmo

- When you activate a transform, you'll notice a set of color-coded arrows or circles appears over the object in the scene;
- This is the Transform gizmo, and you can click-and-drag this gizmo to apply the transformation.



The Move, Rotate, Scale, Rect Transform and Transform Gizmos

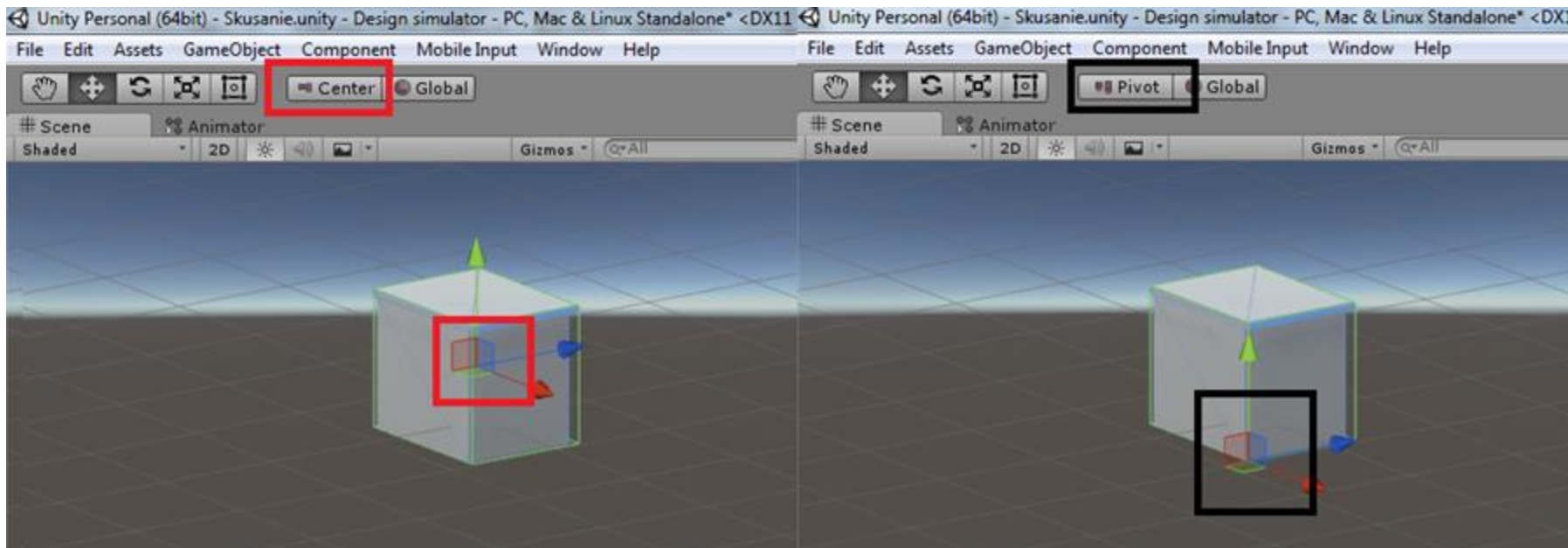
Pivot and Center



- 3D Meshes have a **Pivot**, which defines the coordinate origin of the object
- **Center** moves the coordinate origin of the model at the center (of mass) of the mesh.

Pivot and Center

- By default, Pivot and Center are in the same position.
- However, when they differ, transformation may provide different results.

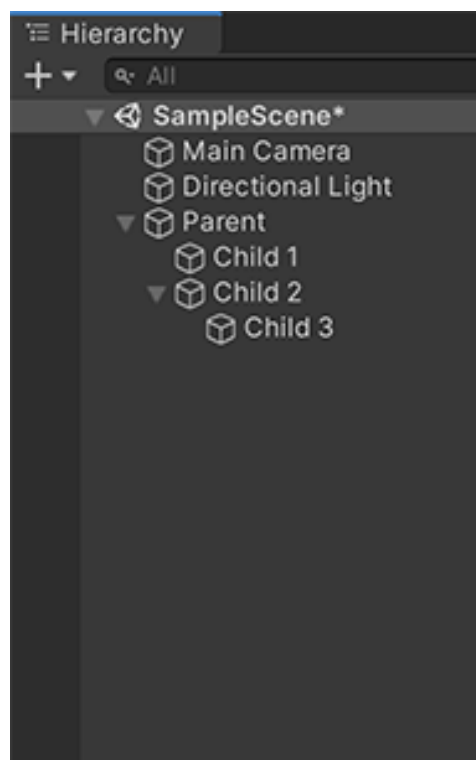


The Hierarchy Tab and Inspector

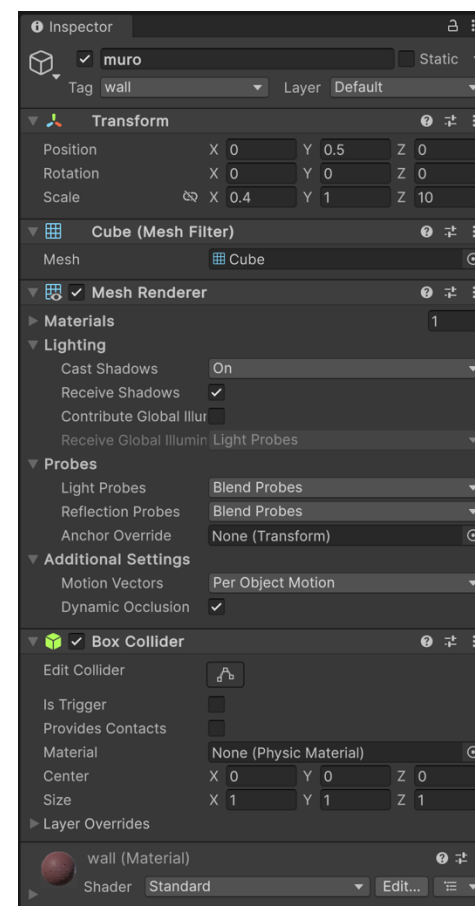
- The **Hierarchy** is a list view with the name of every object in the scene listed, with the names nested together according to their hierarchy linkages in the scene.
- The **Inspector** shows you information about the currently selected object. Select an object and the Inspector is then filled with information about that object. The information shown is pretty much a list of components, and you can even attach or remove components from objects.

Hierarchy and Inspector

Hierarchy

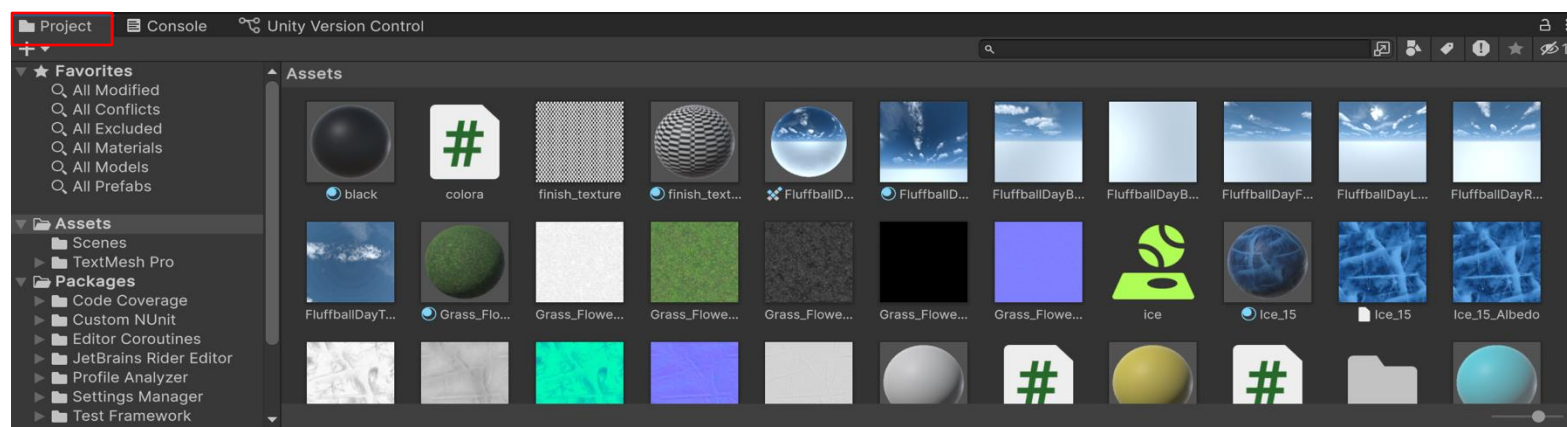


Inspector

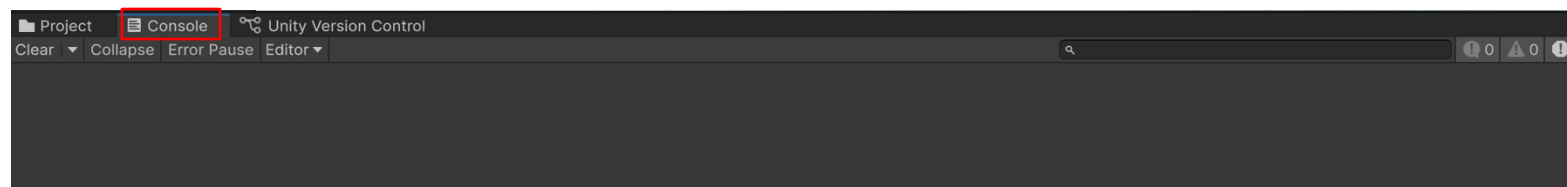


The Project and Console Tab

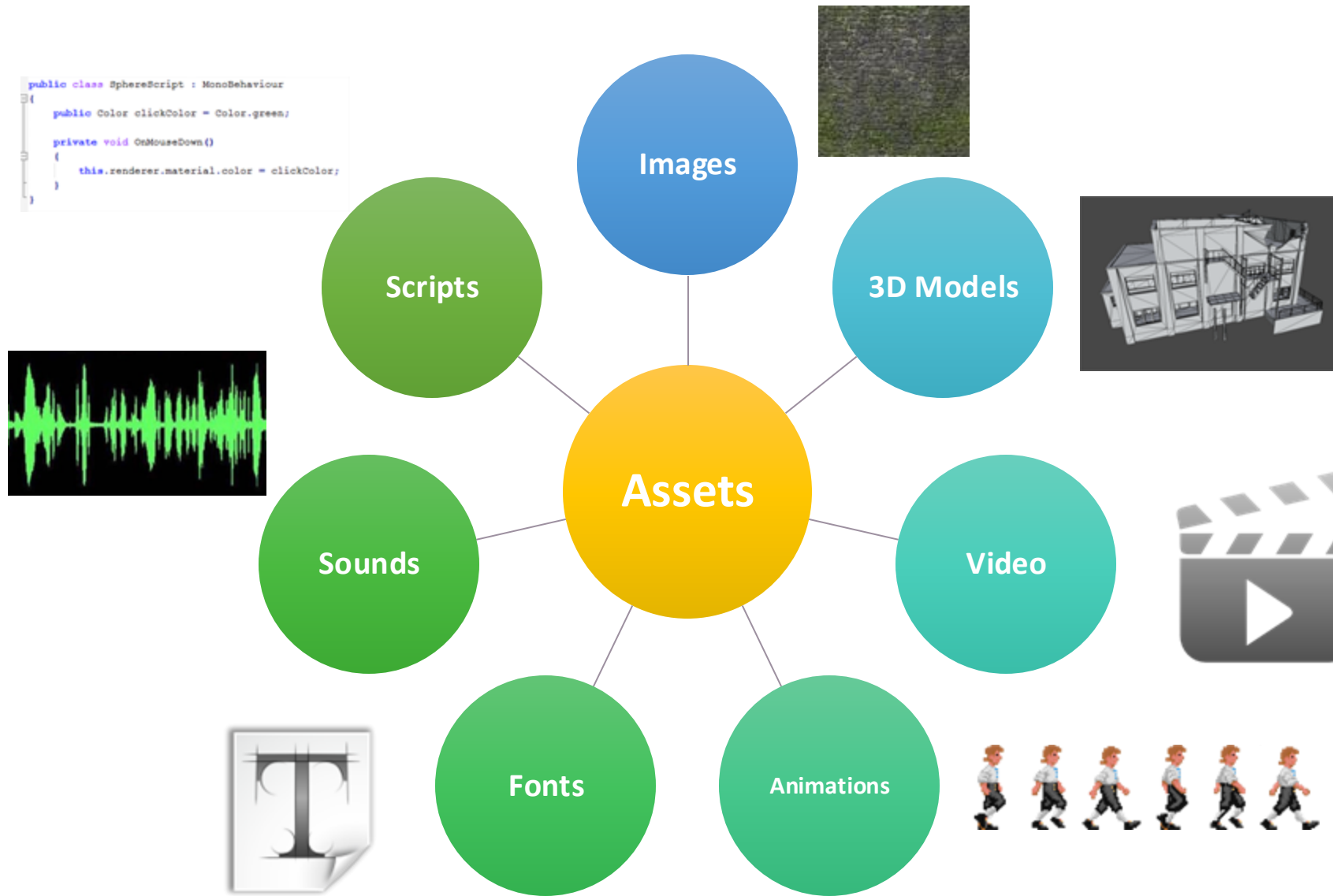
- **Project** shows all the assets (art, code, and so on) in the project



- **The Console** is the place where messages from the code show up.



The Assets



The Assets Store

<https://assetstore.unity.com/>

Publisher Sale is on: **Save 50%** on particle assets and **get three free gifts**

Unity Asset Store Assets Tools Services By Unity Industries Sale

Search for assets

Over 11,000 five-star assets | Rated by 85,000+ customers | Supported by 100,000+ forum members

PUBLISHER SALE

50% off particle assets

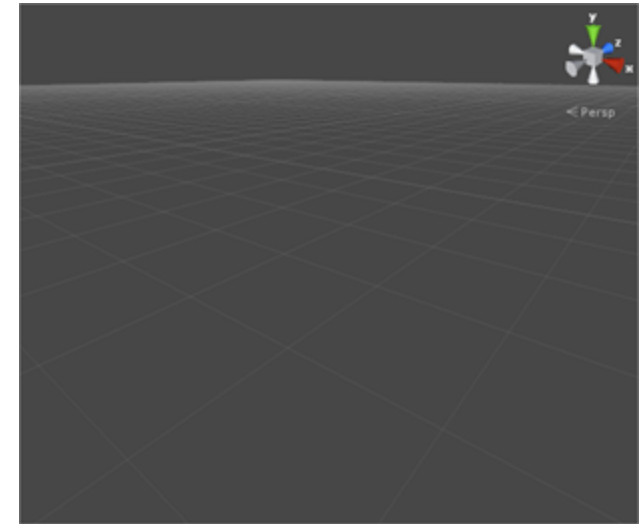
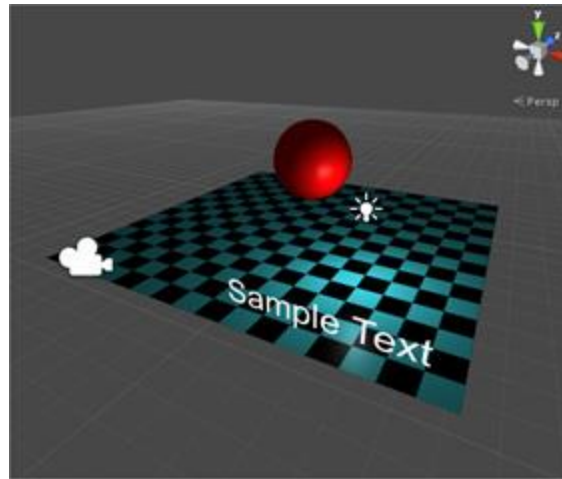
Make a major impact on your project with 50% off particle assets from Kripto289, Archanor VFX, and HOVL Studio, now through October 9.

[Shop now](#)



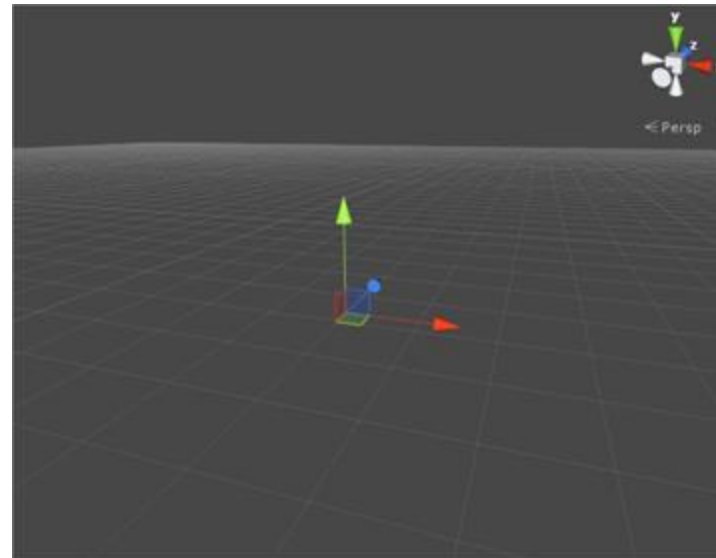
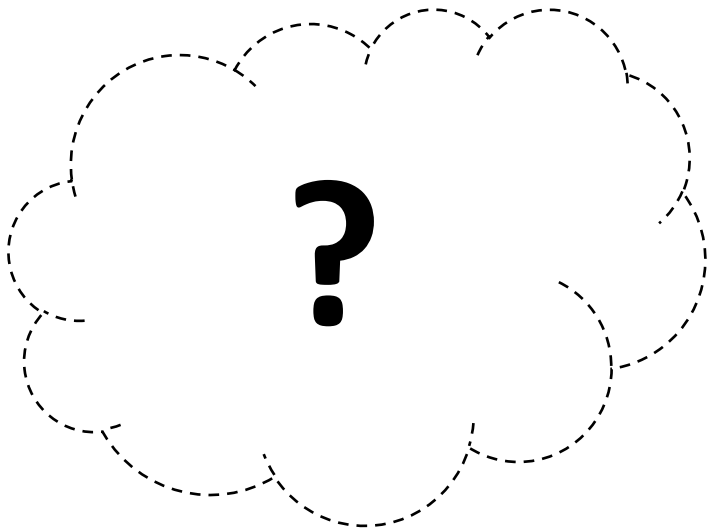
Scene

- Unity games are divided into **scenes**
- **Scenes** are empty spaces where you design the game world
- They can be filled with **game objects**



Game Object

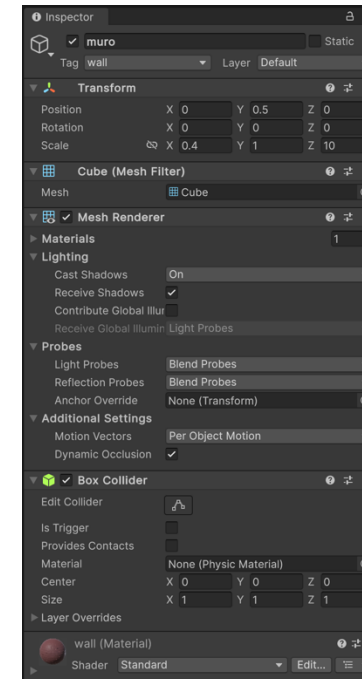
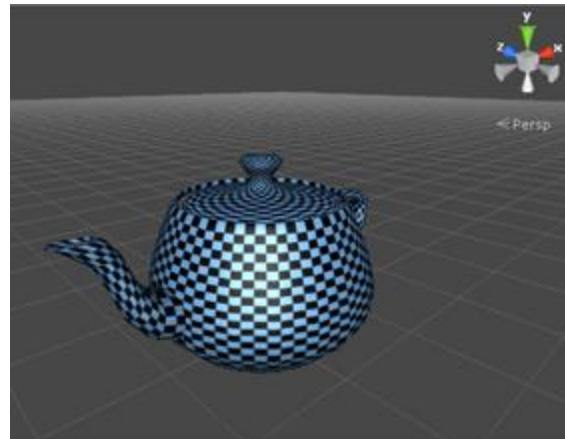
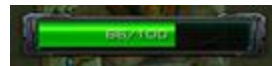
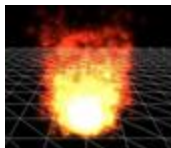
- Everything inside a **scene** is a **game object**
- **Game objects** also start out empty and do nothing...



Game Object



- ...but by adding **components** to them they can become anything!

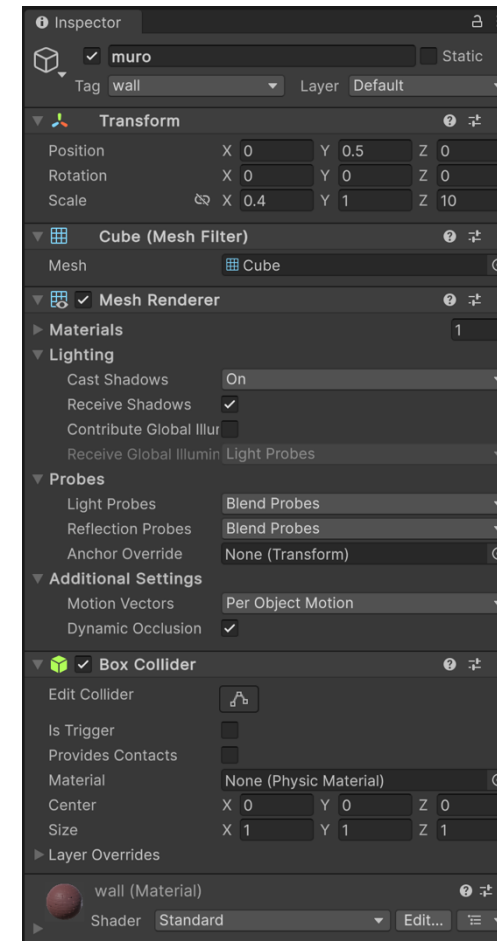


Components

- Each **component** adds a piece of functionality to the **game object**
- The combination of all **components** defines what the **game object** is

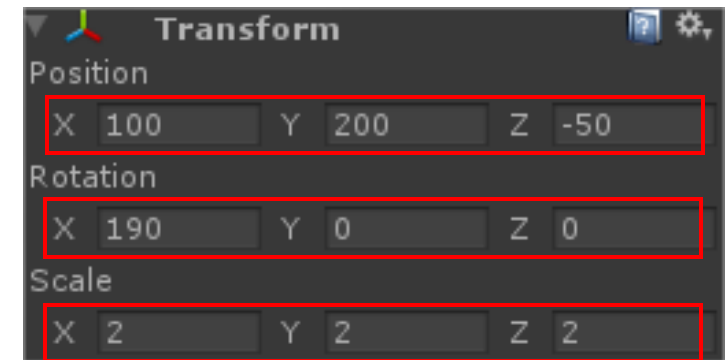
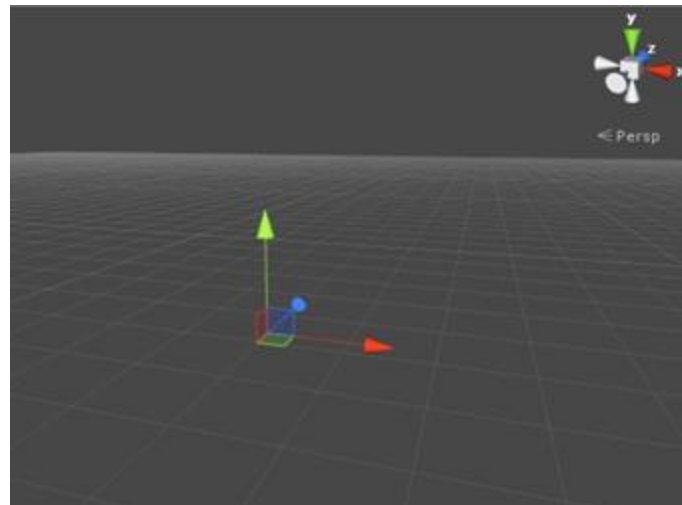
Let's see some examples!

Inspector



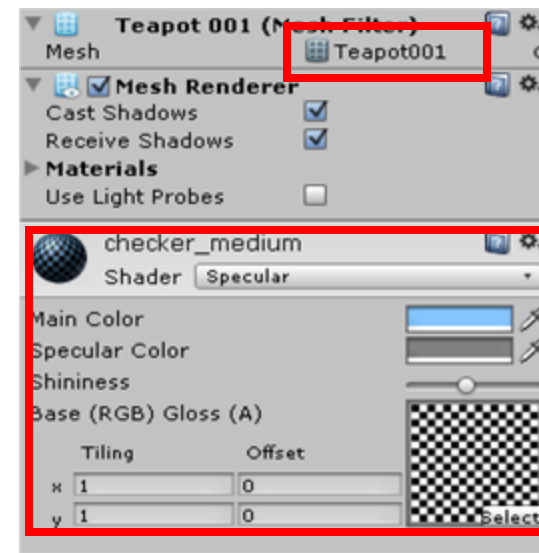
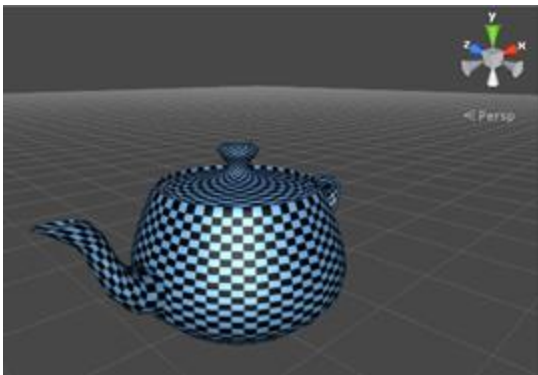
The Transform Component

- The **Transform Component** is one of the **core components** in Unity, automatically attached to every **GameObject**.
- It defines an object **position, rotation, and scale** in the game world.
- Where?
- Which direction?
- How large?



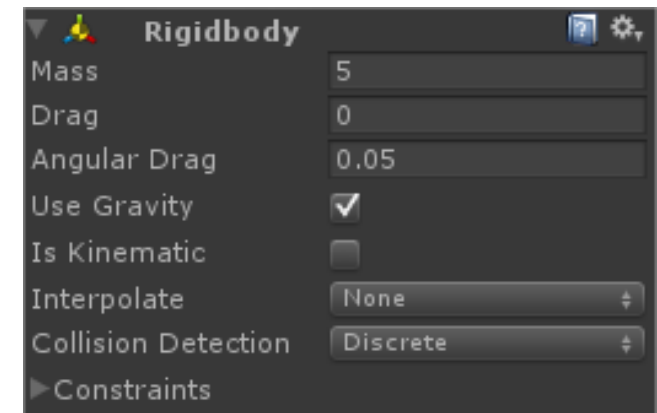
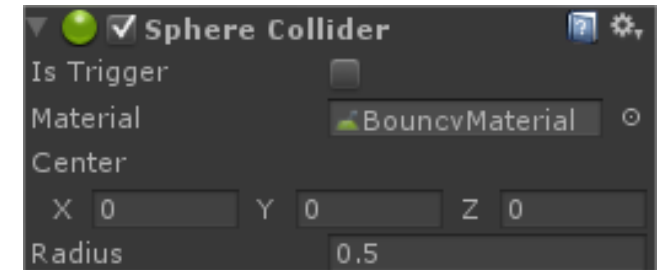
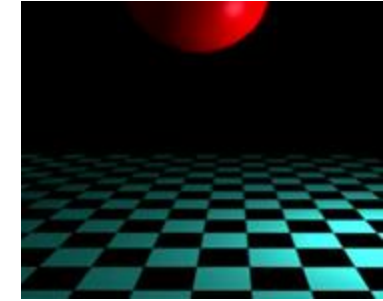
Rendering Component

- **Rendering Component** in Unity is responsible for **displaying objects** on the screen by controlling how they are drawn in the game world.
- It determines an object's **visual appearance**, including its shape, textures, materials, lighting, and shadows.
- What to draw? (mesh filter)
- How to draw? (mesh renderer)



Physics Components

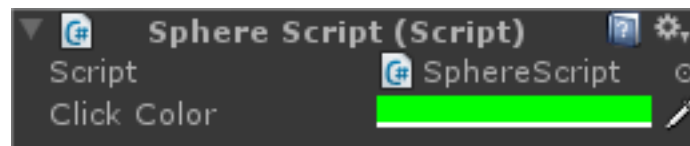
- A **Physics Component** in Unity allows GameObjects to interact with the game world using realistic physical behaviors such as *gravity, collisions, and forces*.
- It is essential for making objects move, collide, and react dynamically based on physics simulations.
- Is solid? (collider)
- Does the object respond to the laws of physics? (rigid body)



The Script Component

- A **Script Component** in Unity is a **custom C# script** attached to a **GameObject** that defines its behavior and logic. It allows developers to control **movement, interactions, animations, game dynamics, and more** through code.

```
1 public class SphereScript : MonoBehaviour
2 {
3     public Color clickColor = Color.green;
4
5     private void OnMouseDown()
6     {
7         this.renderer.material.color = clickColor;
8     }
9 }
```



Other Components

- Light
- Camera
- Text mesh
- Audio listener & source
- Particle system
- Skybox
- ...and many more.

Saving and Exporting a Scene

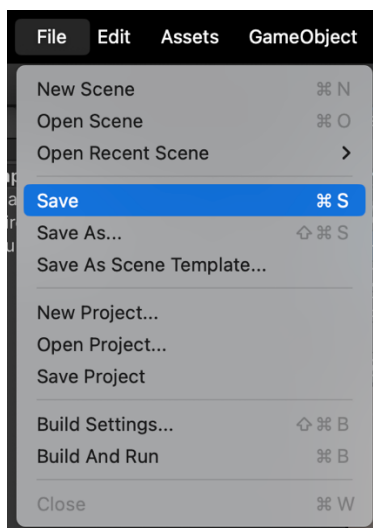


How to save

Unity stores different types of information about your project, and **not all changes are saved the same way**. It's important to understand the difference between **saving a scene** and **saving a project**.

✓ SAVE THE SCENE!

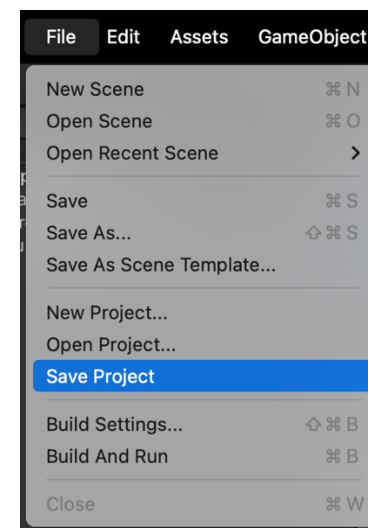
- Go to **File** → **Save** or press **Ctrl + S (Windows)** / **Cmd + S (Mac)**.
- This saves the current scene, including objects, lighting, and layout.



!! If you don't save the scene, any changes will be lost when Unity closes **!!**

✓ SAVE THE PROJECT!

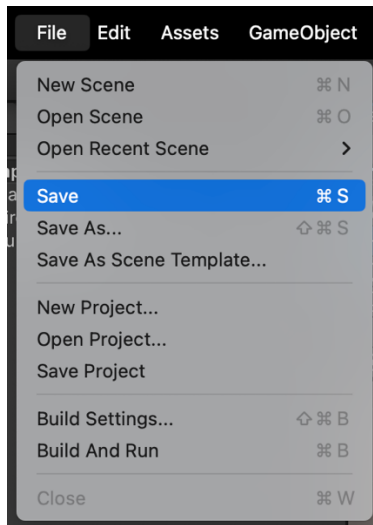
- Go to File → Save Project.
- This saves **project-wide settings**, such as imported assets, prefab modifications, and package settings.



How to save

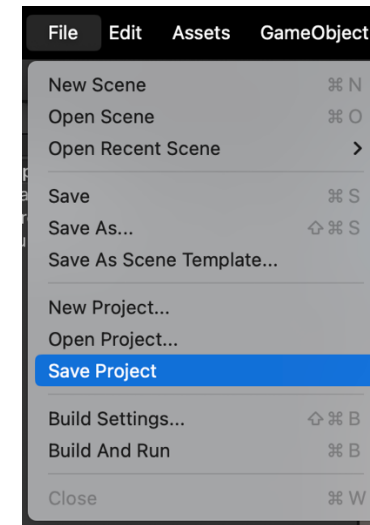
Unity stores different types of information about your project, and **not all changes are saved the same way**. It's important to understand the difference between **saving a scene** and **saving a project**.

✓ SAVE THE SCENE!



!! If you don't save the scene, any changes will be lost when Unity closes !!

✓ SAVE THE PROJECT!



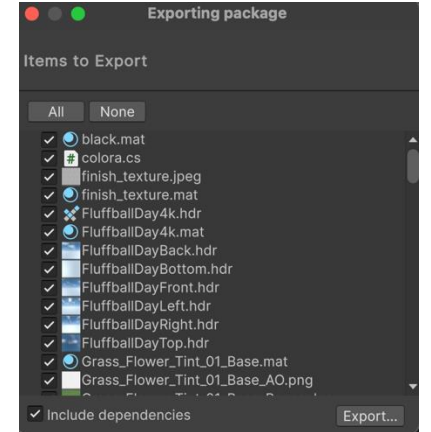
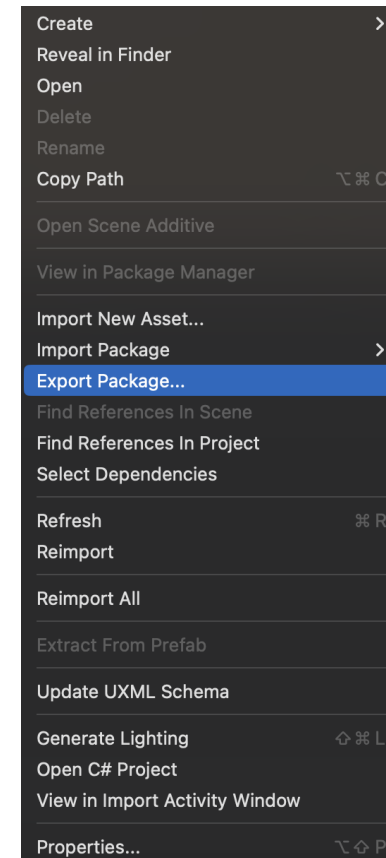
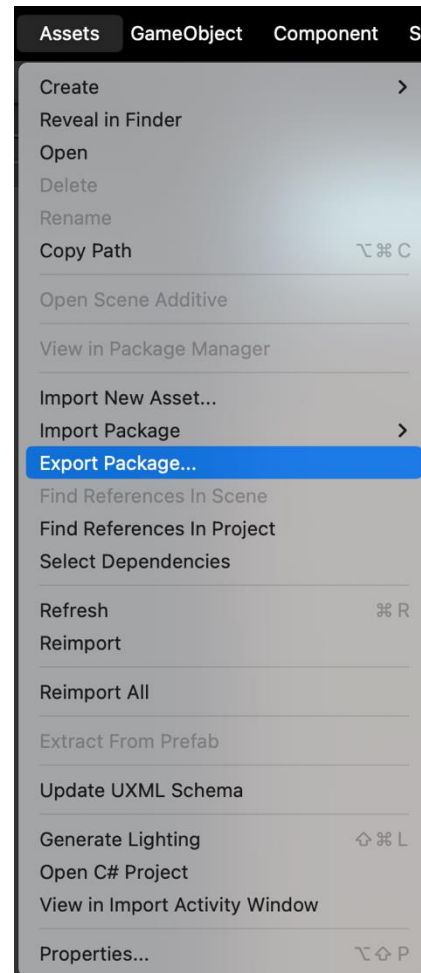
NOTE! Using “Save Project” **does not save** changes to your Scene, only the project-wide changes are saved. You may want to, for instance, save your project but not changes to your scene if you have used a temporary scene to make some changes to a prefab.

Project Export



If you want to transfer your assets into a different project and preserve all the project's information, you can export your **assets** as a **Custom Package**.

1. Open the project you want to export assets from.
2. Choose **Assets > Export Package...** from the menu to bring up the **Exporting Package** dialog box.
3. In the dialog box, select the assets you want to include in the package by clicking on the boxes so they are checked.
4. Leave the **include dependencies** box checked to auto-select any assets used by the ones you have selected.
5. Click on **Export** to bring up File Explorer (Windows) or Finder (Mac) and choose where you want to store your package file. Name and save the package anywhere you like.



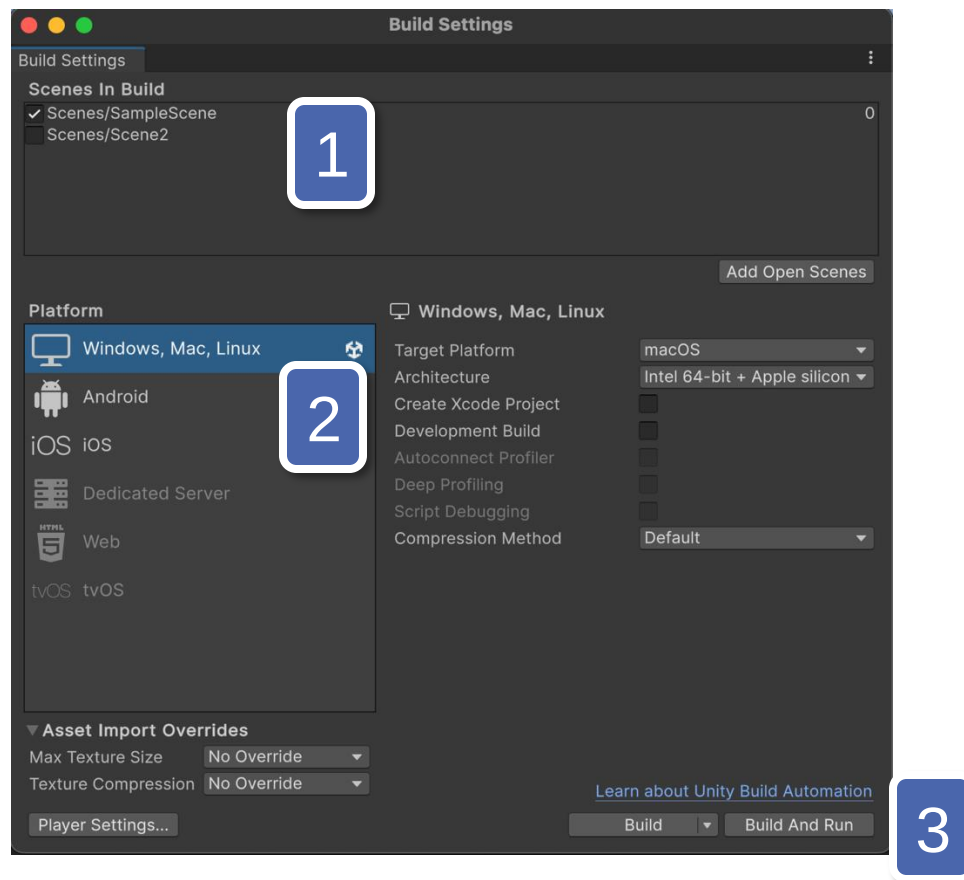


- The screenshot shows the Unity interface with the Hierarchy panel on the left and the Project panel on the right. The Hierarchy panel displays a tree structure with 'Assets' highlighted in red. The Project panel displays a grid of asset thumbnails, including 'Ice_15_AO', 'Ice_15_Height', 'Ice_15_Mask', 'Ice_15_Normal', 'Ice_15_Roughness', 'TextMesh Pro', 'wall', and 'wall2Normals'.



Build

- From the menu bar select File -> Build Settings



1. Select Scenes to Build

2. Select Platform

3. Click Build And Run

4. That's it!